

To make this suitable for a professional project roadmap or a Word-based technical proposal, I have expanded the phases to include specific technical milestones, deliverables, and internal testing checkpoints.

Detailed Project Roadmap: SCK Development

Phase 1: Foundation, Architecture & Environment (Week 1)

Goal: *Establish the "source of truth" for the codebase and ensure both developers have a synchronized local environment.*

- **1.1 Repository & CI/CD Setup (Dev 1):** * Initialize Git with main (stable) and dev (integration) branches.
 - Configure .env templates for database credentials and API keys.
 - Setup FastAPI boilerplate with Uvicorn and Pydantic schemas.
 - **1.2 Database & Data Modeling (Dev 1):** * Design the MySQL schema: Sessions, Messages, GeneratedContexts, and Users.
 - Execute initial migrations using Alembic.
 - **1.3 Extension Scaffolding (Dev 2):** * Develop Manifest V3 structure for Chrome/Edge.
 - Perform DOM-audit: Identify stable CSS selectors for ChatGPT and Claude message containers.
 - **1.4 ML Model Research (Dev 2):** * Benchmark HuggingFace models (e.g., Bart-Large-CNN vs. T5) for summarization latency and token limit efficiency.
-

Phase 2: Data Ingestion & Parser Logic (Weeks 2–3)

Goal: *Build the "Input Layer" to ensure data from any source is converted into a unified JSON format.*

- **2.1 The Parsing Suite (Dev 1):** * Link Parser: Use BeautifulSoup4 to scrape meta-data and chat history from public share URLs.
 - File Parser: Build logic to handle official JSON exports (mapping disparate keys to a unified sck_standard).
 - Raw Text Parser: Implement Regex-based identification to distinguish between "User" and "AI" roles in manual pastes.

- **2.2 Real-Time Capture Engine (Dev 2):** * Implement MutationObserver in the browser extension to "listen" for new message bubbles appearing in the DOM.
 - Develop the local buffer: Store captured messages in chrome.storage.local to prevent data loss on tab refresh.
 - **2.3 Summarization Prototype (Dev 2):** * Create the summarizer.py module.
 - Build logic to chunk long conversations that exceed the ML model's token window (512 or 1024 tokens).
-

Phase 3: Context Engine & Integration (Weeks 4–5)

Goal: Process the gathered data into high-value context documents and connect the Frontend to the Backend.

- **3.1 The Context Strategy Engine (Dev 1):** * Technical Strategy: Prioritize code blocks, error logs, and architectural decisions.
 - Concise Strategy: Focus on high-level goals and the "current state."
 - Creative Strategy: Retain tone, style guides, and narrative world-building elements.
 - **3.2 Platform-Specific Formatting (Dev 1):** * Develop formatters that wrap context in "System Prompt" blocks optimized for Claude's XML tags vs. ChatGPT's Markdown style.
 - **3.3 Streamlit Web Hub (Dev 1):** * Build the dashboard for manual uploads, session history viewing, and one-click "Copy to Clipboard."
 - **3.4 Extension UI & Bridge (Dev 2):** * Design the Popup UI: Add "Record Session," "Generate Context," and status indicators.
 - Establish the fetch() pipeline: Securely send extension data to the FastAPI /parse/extension endpoint.
-

Phase 4: Testing, Refinement & Deployment (Week 6)

Goal: Ensure system stability, handle edge cases, and make the tool publicly available.

- **4.1 Comprehensive Testing (Both):** * Unit Testing: Validate parsers against corrupted or empty JSON files.

- **Integration Testing:** Verify the "Extension-to-Backend-to-Streamlit" data flow.
 - **UX Testing:** Test the extension across different browser themes (Dark/Light mode) to ensure the UI remains readable.
 - **4.2 ML Optimization (Dev 2):** * Optimize model loading times (implementing caching/lazy loading) to ensure the API stays responsive.
 - **4.3 Deployment & Documentation (Dev 1):** * Deploy the FastAPI backend and MySQL database to a cloud provider (Railway/Render/AWS).
 - Finalize the README.md and generate API documentation via Swagger (/docs).
 - **4.4 Release (Both):** * Merge dev into main.
 - Publish the v0.1-alpha release tag on GitHub.
-

Deliverables Summary Table

Phase	Key Deliverable	Owner
Phase 1	System Architecture & Database Schema	Developer 1
Phase 2	Unified Data Parsers & Extension Capture Logic	Both
Phase 3	Context Generation Engine & Streamlit Dashboard	Developer 1
Phase 4	Deployed Application & User Documentation	Developer 1
Phase 4	Finalized ML Summarizer & Chrome Extension	Developer 2

How to use this in Word:

1. Copy the text above.
2. In Word, use Styles (Heading 2 for Phases, Heading 3 for sub-points).

3. The Deliverables Summary Table can be converted into a native Word Table for better visual impact.

Does this level of detail work for your planning, or should we expand on specific technical parts like the ML logic?